

Algorithm for file updates in Python

Project description

Automated IP Allowlist Management for Secure Healthcare Access

This project involves developing a Python-based security tool for a healthcare organization to manage and maintain controlled access to a restricted network containing sensitive patient records.

The organization uses an **allowlist of IP addresses** to determine which employees are permitted to access the secure subnetwork. Due to role changes, offboarding, or policy updates, certain IP addresses must be periodically removed. These are provided in a separate **remove list**.

The goal of this project is to build an algorithm that:

- Reads the current IP allowlist from a file
- Reads or processes a list of IP addresses that should be removed
- Compares both lists to identify matches
- Removes any matching IP addresses from the allowlist
- Updates the file to ensure only authorized IP addresses remain

This automation improves security by ensuring that only approved employees retain access to restricted patient data. It also reduces manual errors and helps the organization maintain compliance with healthcare data protection standards.

The solution demonstrates core Python skills including file handling, list processing, loops, and conditional logic, while applying them to a real-world cybersecurity use case.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

This code sets up the file and data needed for processing. It stores the name of the allow list file in a variable and creates a list of IP addresses that should be removed. Then, it opens the allow list file in read mode so its contents can be accessed and used in the next steps of the program.

Read the file contents

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses= file.read()

# Display `ip_addresses`
print(ip_addresses)

ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

This code prepares and reads an allow list of IP addresses from a file. It first stores the filename in a variable and defines a list of IP addresses that should be removed later. Then it opens the file in read mode and reads all its contents into a variable called `ip_addresses`. Finally, it prints the contents so the current allow list can be viewed.

Convert the string into a list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.168.6.9', '192.168.52.90', '192.168.158.170', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.201.40', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.58.57', '192.168.69.116']
```

This code reads an allow list of IP addresses from a file and prepares it for processing. It first loads the file name and defines a list of IP addresses that need to be removed later. Then it opens the file in read mode and reads all its contents into a variable called `ip_addresses`. After that, it uses `.split()` to convert the data from a single string into a list of individual IP addresses. Finally, it prints the list so the data can be checked in a structured format.

Iterate through the remove list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Display `element` in every iteration
    print(element)

ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

This code reads a file containing an allow list of IP addresses and prepares it for processing. It then converts the file content into a list of individual IP addresses. After that, it loops through each IP address in the list one by one and prints each value separately. This allows the program to display every IP address individually instead of showing them all in a single line or string.

Remove IP addresses that are on the remove list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)

['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9', '192.168.52.90', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.69.116']
```

This code updates an allow list of IP addresses by removing unauthorized entries. It starts by reading the allow list from a file and converting the contents into a list of individual IP addresses. It then loops through each IP address and checks whether it appears in the remove list. If a match is found, that IP address is removed from the allow list. Finally, it prints the updated list showing only the IP addresses that are still allowed access.

Update the file with the revised list of IP addresses

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)

with open(import_file, "r") as file:
    text=file.read()

print(text)

ip_address 192.168.205.12 192.168.6.9 192.168.52.90 192.168.90.124 192.168.186.176 192.168.133.188 192.168.218.219 192.168.52.3
7 192.168.156.224 192.168.60.153 192.168.69.116
```

This code manages and updates a security allow list of IP addresses stored in a file. It starts by reading the file and converting its contents into a list of IP addresses. It then compares each IP address against a remove list and deletes any matches that are no longer authorized. After filtering, the updated list is converted back into a string and written into the same file, replacing the old data. Finally, the file is read again and printed to confirm that only approved IP addresses remain.

Summary

This Python program is used to update a security allow list of IP addresses stored in a text file. It begins by reading the contents of the file and converting the data into a list format so it can be processed more easily.

A separate remove list is defined, containing IP addresses that should no longer have access to the restricted system. The program then loops through the allow list and checks each IP address. If an IP address is found in the remove list, it is removed from the allow list.

After filtering out the unauthorized IPs, the updated list is converted back into a string and written back into the original file, replacing the old data. Finally, the file is read again and printed to confirm the changes.

Overall, the code automates the process of maintaining a secure and up-to-date list of allowed IP addresses, helping improve security and reduce manual work.